Bifrost Name Service: Product-Level Architecture for On-Chain Domain Identity and Governed Resolution

BIFROST FOUNDATION

Bifrost Name Service is an on-chain naming and domain-identity system for the Bifrost ecosystem. It represents names as transferable domain tokens, separates registration authority from ownership state, stores domain records behind bucketed record state, supports resolver-mediated user flows, and includes payment, vault, oracle, meta-transaction, and reward mechanisms around the core registry. This report presents Bifrost Name Service as a product-level technical system rather than as a list of contracts. It describes the operational problem, the domain lifecycle, resolver architecture, state model, payment model, and governance boundaries. Sensitive implementation details such as deployment addresses, private keys, environment variables, and repository path structure are intentionally excluded.

Additional Key Words and Phrases: blockchain infrastructure, identity systems, protocol design

1 Introduction

Blockchain applications need human-meaningful identifiers, but a naming system for a protocol ecosystem is more than a mapping from a string to an address. A domain may be owned as a transferable asset, used as a representative identity, associated with mutable records, renewed over time, parked for later redemption, or operated through delegated and gas-sponsored flows. The naming layer must therefore combine token ownership, registry policy, record storage, resolver interfaces, payment handling, and administrative governance.

Bifrost Name Service addresses this requirement by separating domain identity from operational control. The domain token layer represents ownership and enumerability. The registry layer validates labels, top-level domain policy, expiry, mintability, subdomain authority, parking, renewal, and record updates. Resolver layers expose user-facing and operator-facing flows on top of the registry. Auxiliary layers handle payments, native-coin balances, oracle-backed pricing, relayed execution, and relayer rewards. This separation allows the system to behave like a product rather than a single monolithic naming contract.

This report is scoped to product architecture and safety reasoning. It does not publish deployment addresses, private keys, endpoint URLs, source tree layout, or environment-specific configuration. Component names are used only when they describe product responsibilities that are visible from the inspected local codebase.

Contributions. This report explains Bifrost Name Service as a coherent domain platform. It first frames the operational requirements for a blockchain naming product, including domain ownership, record mutability, expiry, delegated registration, payments, and meta-transaction support. It then describes the core architecture: domain tokens, registry policy, record storage, resolver mediation, vaults, oracle-backed pricing, and relayer execution.

The report also identifies the design boundaries that make the system governable. Administrative authority is represented through named component roles and groups rather than unrestricted public mutation. User-owned domains remain portable, while registry and resolver flows enforce mintability, authorization, expiry, and payment rules. Finally, the report describes the main operating flows and the safety assumptions that should guide production deployment and review.

2 Problem Statement

Bifrost Name Service is motivated by the gap between simple name lookup and a full domain product. A production name service must support asset ownership, human-readable resolution, lifecycle policy, payments, record updates, and delegated execution without exposing privileged mutation paths to arbitrary callers.

Operational Requirements. The first requirement is transferable ownership. A domain should be represented as an ownable asset that wallets, marketplaces, and protocol integrations can understand. Ownership needs

Publication date: July 2026.

to support enumeration, transfer, metadata, and expiry-aware minting. Expired domains must be reclaimable without leaving stale ownership state as permanent obstruction.

The second requirement is registry policy. The platform has to decide which top-level domains are valid, whether a label set is mintable, who can mint reserved or delegated names, and how renewal changes expiry. Subdomains add an additional rule: the parent domain owner should be able to create subordinate names without giving up the platform's global policy controls.

The third requirement is mutable domain data. Domain records need to evolve without changing domain ownership. The system must support record names, record values, record reset behavior, and lookups that are stable for clients but flexible enough for application-specific metadata. Finally, production registration requires payments, fee routing, oracle-assisted price conversion, native-coin custody, and relayed execution so users can interact with the service under practical wallet and gas constraints.

Why Simple Registries Are Insufficient. A minimal registry can map a name to a value, but it does not provide a complete product. It usually lacks expiry, transferability, resolver composition, payment policy, parking, record reset semantics, and relayer accountability. Bifrost Name Service replaces this pattern with a domain-token model and a governed registry/resolver surface.

Table 1. Design contrast between simple registries and Bifrost Name Service.

Concern	Simple registry	Bifrost Name Service
Ownership	Name-value entries	Transferable domain tokens with ownership and enumeration
Lifecycle	Manual overwrite	Minting, expiry, renewal, parking, and redemption
Records	Flat mutable values	Bucketed record state with controlled reset behavior
Access	Contract-level admin	Registry, owner, minter, resolver, and group-based permissions
Payments	External billing	On-chain payment asset handling, fees, vaults, and price conversion
Execution	Direct caller only	Direct calls plus verified relayed execution for domain owners

3 Design Principles

Bifrost Name Service follows five design principles. The first is domain-as-asset identity. A domain is not just a string; it is an owned token with metadata, representative identity, transfer behavior, expiry, and queryability. This gives applications a durable object to integrate with while keeping domain lifecycle rules explicit.

The second principle is registry-mediated mutation. Domain minting, parking, renewal, record updates, and representative changes pass through policy checks. The token layer does not independently decide who is allowed to create names or mutate records. This separation keeps asset ownership distinct from product governance.

The third principle is resolver composition. Resolvers provide user-facing flows over lower-level registry and domain state. A default resolver can combine pricing, asset activation, payment collection, renewal, record updates, and minting into a product surface, while more specialized resolvers can delegate to the same registry primitives.

The fourth principle is state reset without identity loss. Record state is tied to a bucket associated with a domain token. Resetting the bucket changes the record namespace while preserving token identity. This is useful when ownership or operational state changes and old records should no longer be authoritative.

The final principle is governed extension. Payment assets, fee routing, oracle sources, gas behavior, relayer rewards, top-level domains, and minter rights are controlled through explicit administrative surfaces. This design makes the system configurable without publishing unrestricted write authority.

4 System Overview

Bifrost Name Service consists of a domain token layer, registry policy layer, resolver layer, record storage layer, payment and vault layer, oracle layer, and relayed execution layer. The layers are intentionally separated so each major product concern has a clear responsibility and a narrow mutation surface.

The domain token layer stores ownership, expiry, representative identity, token metadata, and enumeration. The registry layer validates name labels, top-level domain status, mintability, subdomain authority, renewal, parking, redemption, and record-update authorization. Resolver components expose higher-level flows for users and operators: minting, renewal, payments, record changes, and lookups. Record storage keeps domain data outside the token ownership path.

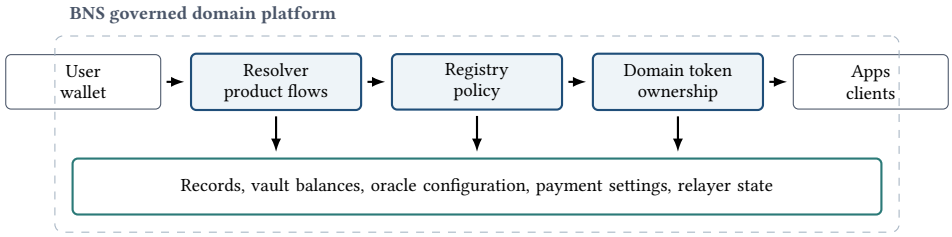

Fig. 1. High-level Bifrost Name Service architecture. The registry controls policy, while domain tokens and records remain separate state surfaces.

The payment and vault layer supports native coin balances, payment collection, fee splitting, and relayer reward reserves. The oracle layer provides asset conversion input for payment assets. The relayed execution layer verifies domain-owner signatures, consumes nonces and gas accounting state, and forwards authorized calls to named platform components.

5 Domain and Registry Model

The domain and registry model is the core of Bifrost Name Service. It separates what a user owns from who may create, renew, park, redeem, and mutate the records associated with that ownership.

Domain Token State. The domain token represents ownership of a name. Token identifiers are derived from labels through a deterministic namehash process. The token layer tracks ownership, expiry, token name, representative domain, bucket identifier, and enumeration. Expiry is important because an occupied token can become mintable again once it is expired.

Representative identity is a separate user-facing concept. A wallet may select one owned domain as its representative identity, and that choice can be updated by the owner through authorized registry or resolver paths. Transfers reset representative state when needed so identity mappings do not silently survive ownership changes in an unsafe way.

Registry Policy. The registry enforces top-level domain policy, label validation, mintability, subdomain authority, renewal, parking, redemption, and record updates. Minting top-level domains is restricted to authorized minter flows. Subdomain minting is permitted when the caller owns the parent domain or has appropriate minting authority. This split allows the product to support user-created subdomains without turning the entire namespace into an unrestricted free-for-all.

Parking and redemption provide operational flexibility. A domain can be minted to a parking vault and later redeemed to a user. This is useful for reserved names, pre-registration workflows, or delayed delivery patterns. Renewal updates expiry while preserving ownership and record continuity.

Record Buckets. Domain records are stored through a key-value record surface. Record names are converted into stable keys, and values are stored under a domain token and a bucket identifier. Resetting the bucket changes the effective record namespace for the domain without changing token ownership. This gives the platform a clean way to invalidate prior records after ownership or operational transitions.

6 Resolver and Payment Surface

Resolvers turn the lower-level registry and domain primitives into usable product workflows. This section explains how user-facing registration, renewal, payment, and record-management flows are composed.

Resolver Responsibilities. The resolver layer provides read and write operations that are more convenient than direct registry interaction. It can query domain ownership, owned domain sets, expiry, representative identity, record values, and mintability. It also coordinates minting, subdomain creation, record updates, representative updates, and record reset actions after checking ownership or platform authorization.

This mediation matters because the registry contains policy primitives, while a product surface needs complete workflows. A resolver can accept payment input, check a deadline, apply pricing, enforce slippage bounds for token payments, and then invoke registry minting or renewal.

Payment Assets and Fees. The default payment model supports active payment assets, per-asset fee configuration, and oracle-assisted conversion. A configured price can be charged in native value or converted into an accepted payment asset. The resolver checks asset activation and fee information before collecting payment and forwarding the platform fee or registration proceeds to the appropriate vault destination.

This design avoids hard-coding a single asset model into the registry. Payment policy can evolve independently from domain ownership and record logic, while the registry still remains the authority for whether a name can be minted or renewed.

Oracle Input. The oracle layer supplies exchange-rate information for payment conversion. The inspected implementation is intentionally conservative: an aggregator reads from configured sources and marks data invalid when a source is missing, non-positive, or stale. Production deployments should treat oracle source selection, update cadence, and fallback behavior as governance-critical configuration.

7 Meta-Transaction and Vault Model

Bifrost Name Service includes a relayed execution model so domain owners can authorize actions without always being the direct transaction sender. This section describes how the forwarder, nonce, gas, vault, and reward concepts fit together.

Forwarded Execution. A forwarded request includes the target component name, call data, domain token identifier, nonce, value, gas budget, deadline, and signature. The forwarder checks that the request is not expired, that the signature matches the domain owner, that the nonce is valid, and that gas accounting rules are satisfied. Only after these checks does it append contextual sender information and call the target component.

Publication date: July 2026.

This model lets a domain owner authorize controlled operations while a relayer submits the transaction. Nonces prevent replay, deadlines bound signature validity, and gas accounting prevents relayers from executing unbounded work against a domain balance.

Native-Coin Domain Balances. The native-coin vault associates balances with domain tokens. Domain owners can deposit, withdraw, and transfer balances between domain identities. The forwarder can draw from a domain's native-coin balance only through its authorized execution path. This connects relayed value transfer to the same ownership model that governs the domain.

Relayer Rewards. The reward model can compensate relayers based on gas used and configured reward parameters. Rewards can be paid from a vault immediately or reserved if insufficient funds are available. The important product property is that relayer economics are represented as explicit state rather than as off-chain convention.

8 Operational Flows

The preceding sections describe the architecture. This section follows the main workflows a product team or user would experience.

Domain Registration. A registration begins with label validation and mintability checks. If the resolver-mediated flow is used, payment asset status, price conversion, fee configuration, payout limits, and deadline are checked before the registry is called. The registry then mints the domain token, sets expiry, stores the token name, and optionally sets the representative domain if the caller and recipient relationship permits it.

Record Update. A record update begins with an ownership or minter authorization check. The registry resolves the domain token, reads the current bucket, converts the record name into a stable key, and writes the record value. Clients later read records through resolver lookup paths that combine token identity, bucket, and record key.

Relayed Domain Action. In a relayed action, a domain owner signs a bounded request. The relayer submits the request to the forwarder. The forwarder validates the signature, owner, deadline, nonce, gas budget, and optional value transfer, then calls the target component with contextual owner data. After execution, gas is consumed from the domain's accounting state and an optional relayer reward is processed.

Fig. 2. Representative BNS workflow. Product flows pass through resolver and registry checks before mutating token or record state.

9 Governance and Safety

Bifrost Name Service is a permissioned product system around user-owned domain assets. Its safety depends on keeping administrative mutation, ownership mutation, payment policy, and relayed execution in separate boundaries.

Access Boundaries. Administrative functions are guarded by named roles, groups, or component authorization. Top-level domain changes, payment-asset configuration, oracle source management, relayer settings, and upgradeable component references should be treated as privileged operations. User-level operations such as record updates and representative selection should remain tied to domain ownership or explicit minter authority.

State Invariants. Several invariants are central. A domain token should have one current owner or be unminted. Expired domains may become mintable again, but active domains should not be overwritten. Record mutation should use the current bucket for the domain. Subdomain minting should require parent ownership or minter authority. Relayed execution should require valid owner signatures, valid nonces, and bounded deadlines.

Sensitive Information Policy. This report intentionally omits deployment addresses, private keys, source tree paths, environment variables, RPC endpoints, and operational secrets. The technical claims are based on the local source behavior, but the report does not expose configuration values that would be unnecessary for a product-level architecture document.

10 Conclusion

Bifrost Name Service is a product-oriented on-chain naming system. Its core strength is the separation of domain ownership, registry policy, resolver workflows, record storage, payment handling, and relayed execution. This lets the platform provide user-facing domain identity while preserving the governance boundaries required for top-level domains, payment assets, oracle inputs, record mutation, and relayer economics.

Publication date: July 2026.